Object-Oriented Programming (OOPS) in Python

The 4 Pillars of Modern Programming

What is OOP?

- Object-Oriented Programming (OOP) is a paradigm based on objects and classes.
- Python supports OOP with simplicity.
- Key idea: Model real-world entities into code.

- Example: Car, Bank, Dog etc.

- Object are real world entities.
- Classes are the group of those entities
- Python treats almost everything as an object that has certain characteristics (properties) and actions it can perform (methods)
- A class is like “Blueprint” or “skeleton” for creating object.
- Oops for 2 purposes: Work with class and object and security.

Other OOP Features in Python

- __init__() – Constructor
- self – Refers to current object (instance) and is used to access variables that belongs to the class.
- super() – Call parent methods
- Types: Single, Multiple, Multilevel, Hierarchical

```
class employee:
    def data_put(self):
        self.id = int(input("Enter the employee id: "))
        self.name = input("Enter the employee name: ")
        self.salary = float(input("Enter the salary: "))
    def  display(self):
        print("Employee id is:", self.id)
        print("Employee Name is:", self.name)
        print("Employee Salary is:", self.salary)
a = employee()
a.data_put()
a.display()
```

4 Pillars of OOPS

- Encapsulation – Hides data using classes/objects
- Abstraction – Shows only essential features
- Inheritance – Reuse code from another class
- Polymorphism – Same method, different behaviors

Class and Object in Python

```
class Car:
    def __init__(self, brand):
        self.brand = brand
    def drive(self):
        print(f"Driving {self.brand}")

c = Car("Honda")
c.drive()
```

Encapsulation

- Encapsulation is the practice of hiding internal object details.

- Example:
- class Bank:
- def __init__(self):
- self.__balance = 1000

- Use getter/setter methods to access data securely.

Inheritance

- Inheritance allows a class to use features of another class.

- Example:
- class Animal:
- def speak(self): print("Animal speaks")

- class Dog(Animal):
- def speak(self): print("Dog barks")

Polymorphism

- Polymorphism means 'many forms'.
- Same method name behaves differently in different classes.

- Example:
- class Bird: def fly(self): print("Flying")
- class Penguin(Bird): def fly(self): print("Can't fly")

Abstraction

- Abstraction hides complex implementation from user.
- Implemented using Abstract Base Classes (ABC):

- from abc import ABC, abstractmethod
- class Shape(ABC):
- @abstractmethod
- def area(self): pass

Summary

- OOP makes code modular, scalable, and reusable.
- Python simplifies OOP with readable syntax.
- Master classes and objects to excel in Python!

Practice Time!

1. Create a class Employee with name, salary, and method display_info.
2. Create a parent class Vehicle and child class Bike.
3. Show polymorphism with an Animal class and subclasses.

QUICK QUIZ

1. What is the purpose of __init__() method in Python?

A. It deletes an object
B. It creates a new module
C. It initializes a newly created object
D. It checks memory leaks

Correct Answer: C

2. Which of the following is not a pillar of OOP?

A. Inheritance
B. Compilation
C. Encapsulation
D. Abstraction

Correct Answer: B

3. What is 'self' in Python classes?

A. A keyword for inheritance
B. Refers to the instance of the class
C. A built-in function
D. None of the above

Correct Answer: B

5. Which access modifier makes a variable private in Python?

A. @private
B. private
C. __ (double underscore prefix)
D. _ (single underscore)

Correct Answer: C

QUICK QUIZ

6. What does polymorphism allow in Object-Oriented Programming?

A. Multiple classes to inherit from one parent class

B. The same function name to behave differently based on the object

C. Data to be stored in private variables

D. Objects to be deleted automatically

Correct Answer: B

Python wasn't named after the snake — it was named after a British comedy group!

Python's creator, **Guido van Rossum**, was a big fan of the British comedy series **"Monty Python's Flying Circus"**. When he started writing the language in the late 1980s, he wanted a name that was short, unique, and a bit mysterious — so he chose **"Python"** as a tribute to the comedy troupe.